

Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- Trees
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

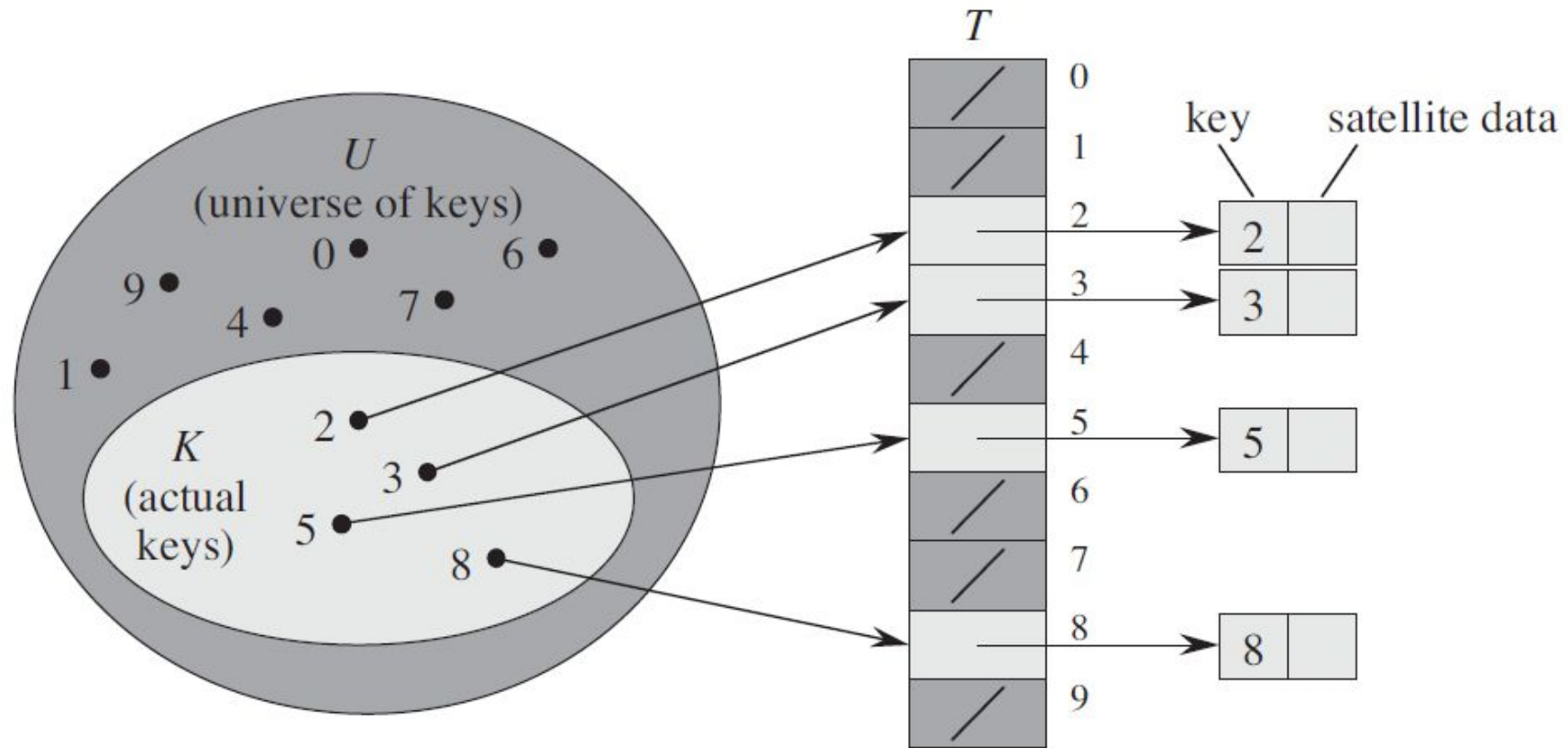
Hash Tables

- A hash table is an effective data structure for implementing dictionaries.
- Searching for an element in a hash table can take as long as searching for an element in a linked list $\Theta(n)$ time in the worst case—in practice, hashing performs extremely well.
- Under reasonable assumptions, the average time to search for an element in a hash table is $O(1)$.

Direct-Address Tables (1)

- Direct addressing is a technique that works well when the universe U of keys is reasonably small. Suppose that an application needs a dynamic set in which each element has a key drawn from the universe $U = \{0, 1, \dots, m - 1\}$, where m is not too large.
- To represent the dynamic set, we use an array, or ***direct-address table***, $T[0..m - 1]$, in which each position, or ***slot***, corresponds to a key in the universe U .

Direct-Address Tables (2)



Direct-Address Tables: Operations (3)

DIRECT-ADDRESS-SEARCH(T, k)

1 **return** $T[k]$

DIRECT-ADDRESS-INSERT(T, x)

1 $T[x.key] = x$

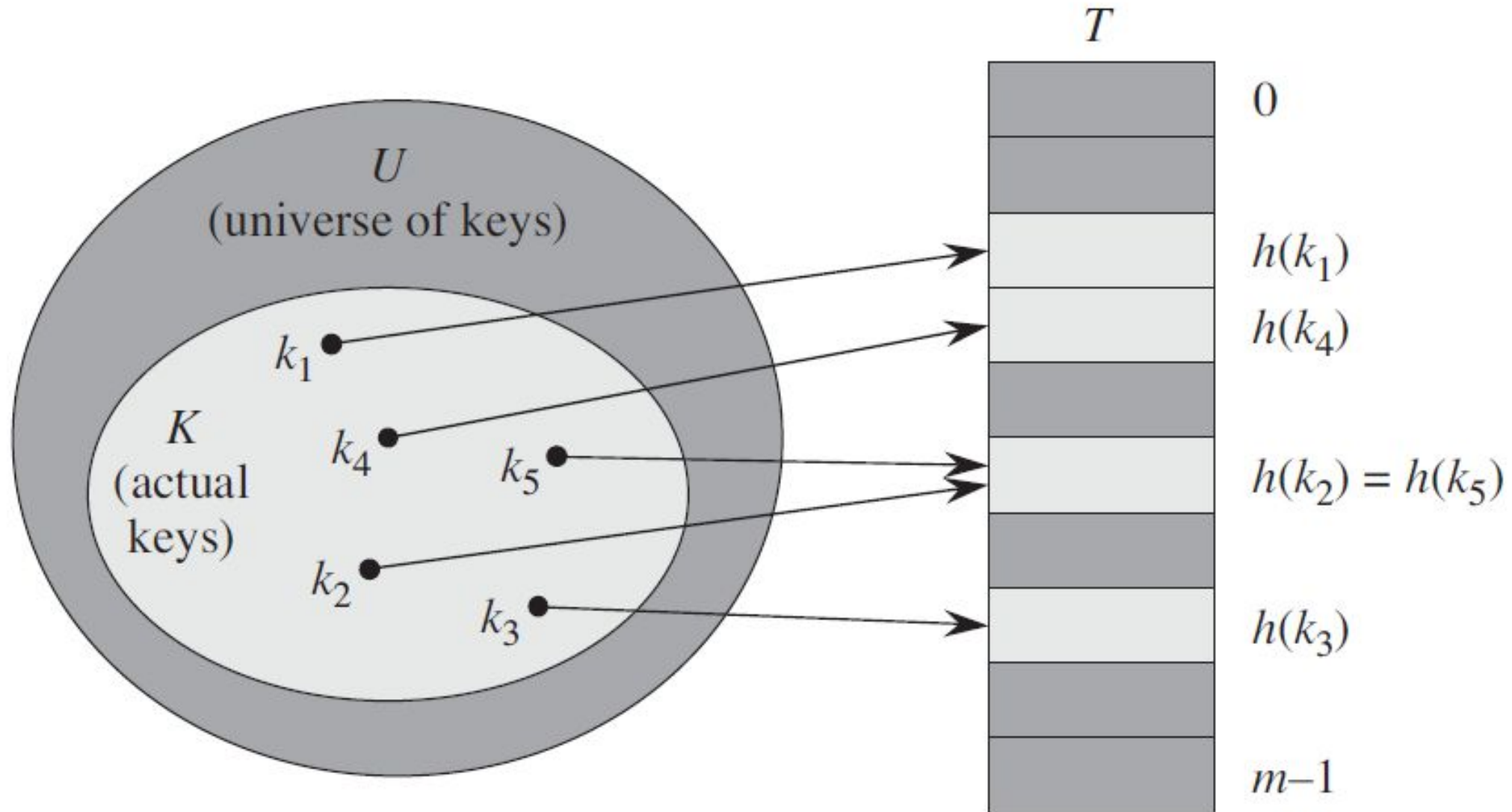
DIRECT-ADDRESS-DELETE(T, x)

1 $T[x.key] = \text{NIL}$

Hash Tables (1)

- With direct addressing, an element with key k is stored in slot k .
- With hashing, this element is stored in slot $h(k)$; a **hash function** h is used to compute the slot from the key k .
- h maps the universe U of keys into the slots of a **hash table**
$$h: U \rightarrow \{0, 1, \dots, m - 1\}$$
$$m \leq |U|$$
- An element with key k **hashes** to slot $h(k)$;
 $h(k)$ is the **hash value** of key k

Hash Tables (2)

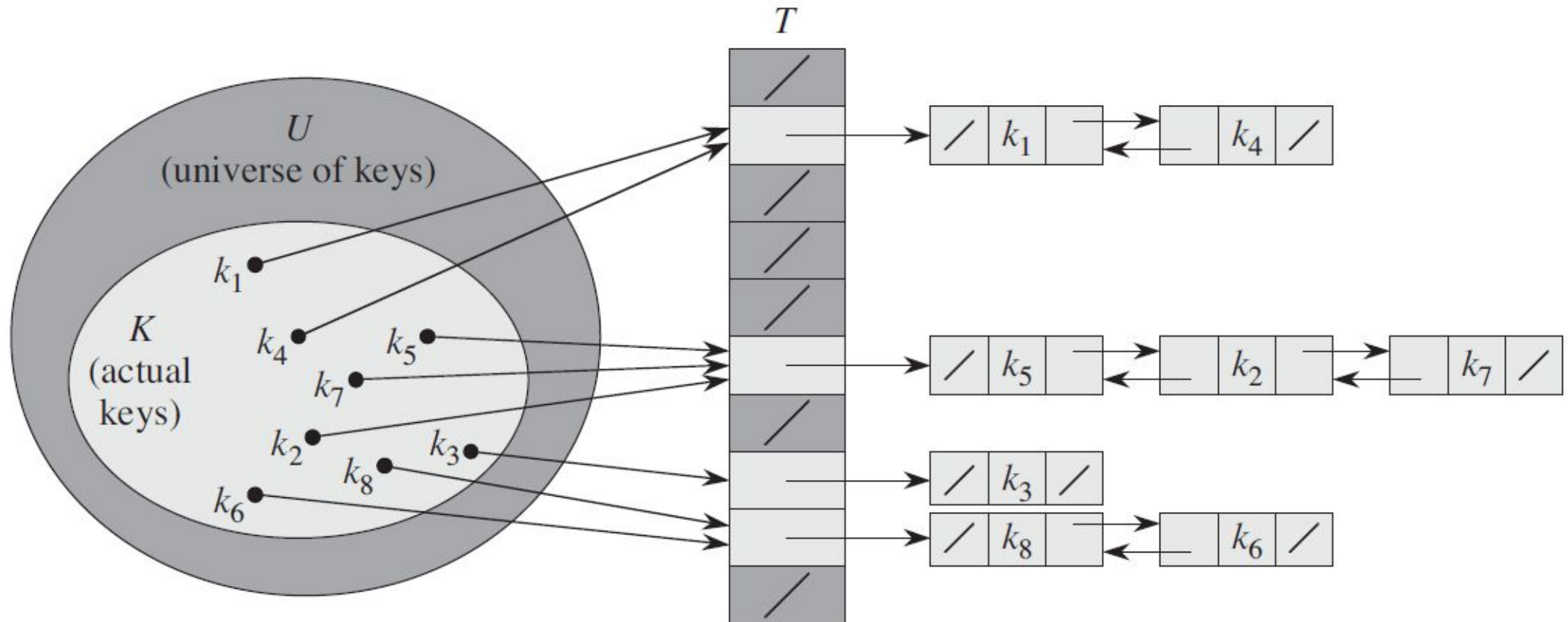


Hash Tables: Collision

- A **collision** is a situation when 2 or more keys hash to the same slot.
- Ideally collisions need to be avoided.
- How to deal with collisions?
 - Choosing suitable hash function h .
 - Making hash function “random” to minimize collisions.
 - Hash function should be deterministic.
 - Chaining

Hash Tables with Chaining: Operations (1)

- In chaining all the elements with same hash value are placed into the same linked list.



Hash Tables with Chaining: Operations (2)

CHAINED-HASH-INSERT(T, x)

- 1 insert x at the head of list $T[h(x.key)]$

CHAINED-HASH-SEARCH(T, k)

- 1 search for an element with key k in list $T[h(k)]$

CHAINED-HASH-DELETE(T, x)

- 1 delete x from the list $T[h(x.key)]$

Analyzing Hashing with Chaining (1)

- Given a hash table T with m slots that stores n elements.
Let $\alpha = n/m$ a **load factor** for T
- Worst-case for hashing with chaining is when all elements hash to the same slot.
- The average-case performance of hashing depends on how well the hash function distributes the set of keys to be stored among the slots, on the average.

Analyzing Hashing with Chaining (2)

- Assume that any given element is equally likely to hash into any of the m slots, independently of where any other element has hashed to. This is called **simple uniform assumption**.
- Let $n_j = T[j].length$: So $n = \sum_{j=0}^{m-1} n_j$
- Expected values for n_j is $E[n_j] = \alpha = n/m$
- Assume time for computing hash value is constant: $O(1)$.
- Search for an element with key k depends linearly on the length $n_{h(k)}$.
 - Unsuccessful search.
 - Successful search.

Analyzing Hashing with Chaining (3)

- **Theorem:** In a hash table in which collisions are resolved by chaining, an unsuccessful search takes average-case time $\Theta(1 + \alpha)$ under the assumption of simple uniform hashing.
- **Proof:** The expected time to search unsuccessfully for a key k is the expected time to search to the end of list $T[h(k)]$, which has expected length $E[n_{h(k)}] = \alpha$. Thus, the expected number of elements examined in an unsuccessful search is α , and the total time required (including the time for computing $h(k)$) is $\Theta(1 + \alpha)$.

Analyzing Hashing with Chaining (4)

- **Theorem:** In a hash table in which collisions are resolved by chaining, a successful search takes average-case time $\Theta(1 + \alpha)$ under the assumption of simple uniform hashing.
- **Proof:**
 - We assume that the element being searched for is equally likely to be any of the n elements stored in the table.
 - Let x_i denote the i th element inserted into the table, for $i = 1, 2, \dots, n$, and let $k_i = x_i.key$.
 - $X_{ij} = I\{h(k_i) = h(k_j)\};$
 - $Pr\{h(k_i) = h(k_j)\} = 1/m;$
 - $E(X_{ij}) = 1/m$

Analyzing Hashing with Chaining (5)

- $$\begin{aligned} E \left[\frac{1}{n} \sum_i^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] &= \frac{1}{n} \sum_i^n \left(1 + \sum_{j=i+1}^n E(X_{ij}) \right) = \\ \frac{1}{n} \sum_i^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) &= \frac{1}{n} \sum_i^n \left(1 + \frac{n-i}{m} \right) = \frac{\sum_{i=1}^n m + \sum_{i=1}^n n - \sum_{i=1}^n i}{nm} \\ &= \frac{nm + n^2 + \frac{n(n+1)}{2}}{nm} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n} \end{aligned}$$

Total time is: $\Theta(1 + \alpha)$

Hash functions

- A good hash function satisfies (approximately) the assumption of simple uniform hashing.
- Most hash functions assume that the universe of keys is the set of natural numbers.
 - if the keys are not natural numbers, we find a way to interpret them as natural numbers
 - For example, a character string can be interpreted as an integer expressed in suitable radix notation.

Hash functions: The Division Method

- In the division method for creating hash functions, we map a key k into one of m slots by taking the remainder of k divided by m

$$h(k) = k \bmod m$$

- Certain values of m are not good choices.
 - m should not be a power of 2
 - a prime not too close to an exact power of 2 is often a good choice

Hash functions: The Multiplication Method

- The multiplication method for creating hash functions operates in two steps.
 - multiply the key k by a constant A in the range $0 < A < 1$ and extract the fractional part of kA
 - multiply this value by m and take the floor of the result
$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$



Thank You